

✓ Load the Iris Dataset

First, we'll load the classic Iris dataset using `scikit-learn` and convert it into a pandas DataFrame for easier manipulation and analysis.

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris

# Load the Iris dataset
iris = load_iris()

# Create a DataFrame from the data
# The feature names are available in iris.feature_names
# The target names are available in iris.target_names (mapping target integers to species names)
df_iris = pd.DataFrame(data=iris.data, columns=iris.feature_names)

# Add the target variable (species) to the DataFrame
df_iris['species'] = iris.target

# Map target integers to species names for better readability
df_iris['species'] = df_iris['species'].map(lambda x: iris.target_names[x])

print("Iris dataset loaded successfully.")
```

Iris dataset loaded successfully.

```
# Display the first 5 rows of the DataFrame
print("First 5 rows of the Iris dataset:")
display(df_iris.head())

# Display information about the DataFrame (columns, non-null counts, data types)
print("\nDataFrame Information:")
df_iris.info()

# Display descriptive statistics
print("\nDescriptive Statistics:")
display(df_iris.describe())
```

First 5 rows of the Iris dataset:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa



DataFrame Information:

```
<class 'pandas.core.frame.DataFrame'>
```

RangeIndex: 150 entries, 0 to 149

Data columns (total 5 columns):

#	Column	Non-Null Count	Dtype
0	sepal length (cm)	150 non-null	float64
1	sepal width (cm)	150 non-null	float64
2	petal length (cm)	150 non-null	float64
3	petal width (cm)	150 non-null	float64
4	species	150 non-null	object

dtypes: float64(4), object(1)

memory usage: 6.0+ KB

Descriptive Statistics:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
count	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.057333	3.758000	1.199333
std	0.828066	0.435866	1.765298	0.762238
min	4.300000	2.000000	1.000000	0.100000
25%	5.100000	2.800000	1.600000	0.300000
50%	5.800000	3.000000	4.350000	1.300000
75%	6.400000	3.300000	5.100000	1.800000
max	7.900000	4.400000	6.900000	2.500000

```
from sklearn.model_selection import train_test_split
```

```
# Separate features (X) and target (y)
```

```
X = df_iris.drop('species', axis=1)
```

```
y = df_iris['species']
```

```
# Split the dataset into training and testing sets
```

```
# We'll use a test size of 30% and a random state for reproducibility
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, ran
```

```
print(f"Training set size: {len(X_train)} samples")
print(f"Test set size: {len(X_test)} samples")
```

```
Training set size: 105 samples
Test set size: 45 samples
```

```
from sklearn.naive_bayes import GaussianNB

# Initialize the Gaussian Naive Bayes classifier
gnb = GaussianNB()

# Train the model using the training data
gnb.fit(X_train, y_train)

print("Gaussian Naive Bayes model trained successfully.")
```

```
Gaussian Naive Bayes model trained successfully.
```

```
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Make predictions on the test set
y_pred = gnb.predict(X_test)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f"Model Accuracy: {accuracy:.4f}\n")

# Display classification report
print("Classification Report:\n")
print(classification_report(y_test, y_pred))

# Display confusion matrix
print("Confusion Matrix:\n")
conf_matrix = confusion_matrix(y_test, y_pred)
display(pd.DataFrame(conf_matrix, index=gnb.classes_, columns=gnb.classes_))

# Visualize the confusion matrix
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues',
            xticklabels=gnb.classes_, yticklabels=gnb.classes_)
plt.title('Confusion Matrix')
plt.xlabel('Predicted Species')
plt.ylabel('True Species')
plt.show()
```

Model Accuracy: 0.9778

Classification Report:

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	19
versicolor	1.00	0.92	0.96	13
virginica	0.93	1.00	0.96	13
accuracy			0.98	45
macro avg	0.98	0.97	0.97	45
weighted avg	0.98	0.98	0.98	45

Confusion Matrix:

	setosa	versicolor	virginica
setosa	19	0	0
versicolor	0	12	1
virginica	0	0	13

